

FC7xxx LIN Integration Manual

Rev.0.3

Revision History

Revision	Date	Changes
0.1	2023/07/14	Initial release for MCAL V0.1.0
0.3	2023/10/20	Updated for MCAL V0.3.0

Table of Contents

Chapter 1 Introduction	4
1.1 Introduction	4
Chapter 2 Building	5
2.1 Dependencies on Other Modules.....	5
2.2 Files Required for Compile	5
2.3 Add Plug-ins	6
Chapter 3 Memory.....	7
3.1 Sections in Memory Map	7
Chapter 4 Exclusive Area.....	9
Chapter 5 Interrupt Service Routine (ISR).....	10
Chapter 6 Error Report.....	11
6.1 Det.....	11
6.2 Dem.....	11
Chapter 7 Function Calls.....	12
7.1 Function Calls during Startup	12
7.2 Function Calls during Shutdown	12
7.3 Function Calls during Wake-up	12
Chapter 8 Other Requirements	13
8.1 Notification, Callback, Callout	13
8.2 Macros	13
Chapter 9 Integration Steps	14

Chapter 1 Introduction

1.1 Introduction

This integration manual describes the integration requirements for the LIN module.

Chapter 2 Building

2.1 Dependencies on Other Modules

Module configuration dependency

- Mcu: This module provides the clock reference point for LIN module
- Port: This module provides the port configurations if want output LIN signal through pin.
- Common: This module is the basic module which used to choose the chip.
- Ecuc: This module is required for selecting the reference to the partition
- EcuM: is required for selecting the reference to the wakeup source for every Lin controller.
- Det: is required for signaling the default error detection (parameters out of range, null pointers, etc)
- Dem: is required for signaling the production error detection (hardware failure, etc).

Module build dependency

- Common: This module provides common type for other modules.
- Rte: This module provides APIs to protect/unprotect some parts of code from interrupts (Exclusive Areas).
- Det: This module is necessary for tracing Development errors.
- Dem: This module is necessary for enabling Development error detection.
- Linif: This module provides the Linif wakeup confirmation callback function.
- EcuM: This module provides the EcuM callback functions when wakeup support is enabled.

Module initialization dependency

- Mcu: To use the LIN module, the user needs to initialize the MCU module first
- Port: To output LIN signals, the user needs to initialize PORT module first and assign the UART pins.

2.2 Files Required for Compile

Lin module files:

- _MCAL/Src/Lin/Src/Lin.c
- _MCAL/Src/Lin/Src/Lin_FCUart.c
- _MCAL/Src/Lin/Src/Lin_Irq.c
- _MCAL/Src/Lin/include/Lin.h
- _MCAL/Src/Lin/include/Lin_FCUart.h
- _MCAL/Src/Lin/include/Lin_Lin_FCUart_RegOps.h
- _MCAL/Src/Lin/include/Lin_GeneralTypes.h
- _MCAL/Src/Lin/include/Lin_MemMap.h
- _MCAL/Src/Lin/include/FCUart_Reg.h
- _MCAL/Src/Lin/include/Lin_version.h
- _MCAL_generate/src/Lin_PBcfg.c
- _MCAL_generate/src/Lin_Cfg.c
- _MCAL_generate/include/Lin_PBcfg.h
- _MCAL_generate/include/Lin_Cfg.h

Common module files:

- _MCAL/Src/Common/include/Mcal.h
- _MCAL/Src/Common/include/Std_Types.h
- _MCAL/Src/Common/include/Platform_Types.h
- _MCAL/Src/Common/include/Compiler.h
- _MCAL/Src/Common/include/Compiler_Cfg.h
- _MCAL/Src/Common/include/CompilerDefinition.h

Det module files:

- Det.h

Rte module files:

- SchM_Lin.h

Dem module files:

- Dem.h

Linif module files:

- LinIf.h
- LinIf_Cbk.h

EcuM module files:

- EcuM_Cbk.h

2.3 Add Plug-ins

LIN module plug-ins are developed for EB tresos Studio, so, to use LIN plug-ins on the EB tresos Studio, the user needs to add the LIN module to the EB plug-ins folder first.

- 1) Copy the LIN module(_MCAL/EB_Plugins/eclipse/plugins/Lin) folder to EB tresos plug-ins (EB/tresos/plugins/) folder.
- 2) Set the LIN module output location folder for the generated source file and header files.
- 3) Use EB tresos to configure the LIN module and generate source and header files.

Chapter 3 Memory

3.1 Sections in Memory Map

Section Name	Section Type	Description
LIN_START_SEC_CONFIG_DATA_8 LIN_STOP_SEC_CONFIG_DATA_8 LIN_START_SEC_CONFIG_DATA_16 LIN_STOP_SEC_CONFIG_DATA_16 LIN_START_SEC_CONFIG_DATA_32 LIN_STOP_SEC_CONFIG_DATA_32	Variables	These are all the sections used for variables which have to be aligned to 8/16/32 bit. These variables are initialized by startup code (data).
LIN_START_SEC_CONFIG_DATA_UNSPECIFIED LIN_STOP_SEC_CONFIG_DATA_UNSPECIFIED	Configuration Data	Start and stop of Memory Section for Config Data.
LIN_START_SEC_CONST_BOOLEAN LIN_STOP_SEC_CONST_BOOLEAN LIN_START_SEC_CONST_8 LIN_STOP_SEC_CONST_8 LIN_START_SEC_CONST_16 LIN_STOP_SEC_CONST_16 LIN_START_SEC_CONST_32 LIN_STOP_SEC_CONST_32 LIN_START_SEC_CONST_UNSPECIFIED LIN_STOP_SEC_CONST_UNSPECIFIED	Variables	These are all the sections used for variables which have to be aligned to 8/16/32 bit or boolean. These variables are read only (rodata).
LIN_START_SEC_CODE LIN_STOP_SEC_CODE LIN_START_SEC_RAMCODE LIN_STOP_SEC_RAMCODE LIN_START_SEC_CODE_AC LIN_STOP_SEC_CODE_AC	Code	Start and stop of memory Section for Code (text).
LIN_START_SEC_VAR_NO_INIT_BOOLEAN LIN_STOP_SEC_VAR_NO_INIT_BOOLEAN LIN_START_SEC_VAR_NO_INIT_8 LIN_STOP_SEC_VAR_NO_INIT_8 LIN_START_SEC_VAR_NO_INIT_16 LIN_STOP_SEC_VAR_NO_INIT_16 LIN_START_SEC_VAR_NO_INIT_32 LIN_STOP_SEC_VAR_NO_INIT_32 LIN_START_SEC_VAR_NO_INIT_UNSPECIFIED LIN_STOP_SEC_VAR_NO_INIT_UNSPECIFIED	Variables	These are all the sections used for variables which have to be aligned to 8/16/32 bit. These variables are never cleared and never initialized by startup code (bss).
LIN_START_SEC_VAR_INIT_BOOLEAN LIN_STOP_SEC_VAR_INIT_BOOLEAN LIN_START_SEC_VAR_INIT_8 LIN_STOP_SEC_VAR_INIT_8	Variables	These are all the sections used for variables which have to be aligned to 8/16/32 bit. These

Section Name	Section Type	Description
LIN_START_SEC_VAR_INIT_16 LIN_STOP_SEC_VAR_INIT_16 LIN_START_SEC_VAR_INIT_32 LIN_STOP_SEC_VAR_INIT_32 LIN_START_SEC_VAR_INIT_UNSPECIFIED LIN_STOP_SEC_VAR_INIT_UNSPECIFIED		variables are initialized by startup code (data).

Chapter 4 Exclusive Area

LIN module using the services of Schedule Manger (SchM) for entering and exiting critical regions.

The following critical regions are used in the LIN driver:

- Lin_FCuart.c:
 - LIN_EXCLUSIVE_AREA_00 is used in Lin_LL_ChannelInit.
 - LIN_EXCLUSIVE_AREA_01 is used in Lin_LL_SendHeader.
 - LIN_EXCLUSIVE_AREA_02 is used in Lin_LL_SendHeader.
 - LIN_EXCLUSIVE_AREA_03 is used in Lin_LL_SendResponse.
 - LIN_EXCLUSIVE_AREA_04 is used in Lin_LL_GoToSleep.
 - LIN_EXCLUSIVE_AREA_05 is used in Lin_LL_GoToSleep.
 - LIN_EXCLUSIVE_AREA_06 is used in Lin_LL_GoToSleepInternal.
 - LIN_EXCLUSIVE_AREA_07 is used in Lin_LL_GoToSleepInternal.
 - LIN_EXCLUSIVE_AREA_08 is used in Lin_LL_WakeUp.
 - LIN_EXCLUSIVE_AREA_09 is used in Lin_LL_WakeupInternal.
 - LIN_EXCLUSIVE_AREA_10 is used in Lin_LL_TxRxInterruptHandler.
 - LIN_EXCLUSIVE_AREA_11 is used in Lin_LL_TxRxInterruptHandler.
 - LIN_EXCLUSIVE_AREA_12 is used in Lin_LL_RxDataReadyHandler.
 - LIN_EXCLUSIVE_AREA_13 is used in Lin_LL_InterruptSourceSlave.
 - LIN_EXCLUSIVE_AREA_14 is used in Lin_LL_RxDataReadyHandler.
 - LIN_EXCLUSIVE_AREA_15 is used in Lin_LL_RxDataReadyHandler.
 - LIN_EXCLUSIVE_AREA_16 is used in Lin_LL_RxDataReadyHandler.
 - LIN_EXCLUSIVE_AREA_17 is used in Lin_LL_RxDataReadyHandler.
 - LIN_EXCLUSIVE_AREA_18 is used in Lin_LL_InterruptSourceSlave.
 - LIN_EXCLUSIVE_AREA_19 is used in Lin_LL_TxRxInterruptHandler.
 - LIN_EXCLUSIVE_AREA_20 is used in Lin_LL_IdleInterruptHandler.
 - LIN_EXCLUSIVE_AREA_21 is used in Lin_LL_IdleInterruptHandler.
 - LIN_EXCLUSIVE_AREA_22 is used in Lin_LL_ShutDownReset.
 - LIN_EXCLUSIVE_AREA_23 is used in Lin_LL_ErrorInterruptHandler.
 - LIN_EXCLUSIVE_AREA_24 is used in Lin_LL_RxDataReadyHandler.
 - LIN_EXCLUSIVE_AREA_25 is used in Lin_LL_ChannelInit.

Chapter 5 Interrupt Service Routine (ISR)

Instance	Interrupt Name	IRQ Number (NVIC Interrupt ID)
FCUART0	FCUART0_RxTx_IRQHandler	100
FCUART1	FCUART1_RxTx_IRQHandler	101
FCUART2	FCUART2_RxTx_IRQHandler	102
FCUART3	FCUART3_RxTx_IRQHandler	103
FCUART4	FCUART4_RxTx_IRQHandler	104
FCUART5	FCUART5_RxTx_IRQHandler	105
FCUART6	FCUART6_RxTx_IRQHandler	106
FCUART7	FCUART7_RxTx_IRQHandler	107
FCUART8	FCUART8_RxTx_IRQHandler	108
FCUART9	FCUART9_RxTx_IRQHandler	109
FCUART10	FCUART10_RxTx_IRQHandler	110
FCUART11	FCUART11_RxTx_IRQHandler	111
FCUART12	FCUART12_RxTx_IRQHandler	112
FCUART13	FCUART13_RxTx_IRQHandler	113
FCUART14	FCUART14_RxTx_IRQHandler	114
FCUART15	FCUART15_RxTx_IRQHandler	115
FCUART16	FCUART16_RxTx_IRQHandler	148
FCUART17	FCUART17_RxTx_IRQHandler	149

Chapter 6 Error Report

6.1 Det

Function Name	Error Type
Lin_Init	LIN_E_STATE_TRANSITION; LIN_E_INVALID_POINTER; LIN_E_ALREADY_INITIALIZED;
Lin_CheckWakeup	LIN_E_UNINIT; LIN_E_INVALID_CHANNEL;
Lin_GetStatus	LIN_E_UNINIT; LIN_E_INVALID_CHANNEL; LIN_E_PARAM_POINTER;
Lin_GoToSleep	LIN_E_UNINIT; LIN_E_INVALID_CHANNEL;
Lin_GoToSleepInternal	LIN_E_UNINIT; LIN_E_INVALID_CHANNEL;
Lin_SendFrame	LIN_E_UNINIT; LIN_E_INVALID_CHANNEL; LIN_E_PARAM_POINTER;
Lin_Wakeup	LIN_E_UNINIT; LIN_E_INVALID_CHANNEL; LIN_E_STATE_TRANSITION;
Lin_WakeupInternal	LIN_E_UNINIT; LIN_E_INVALID_CHANNEL; LIN_E_STATE_TRANSITION;

6.2 Dem

Function Name	Error Type
Lin_GoToSleep	DEM_EVENT_STATUS_FAILED; DEM_EVENT_STATUS_PASSED;
Lin_GoToSleepInternal	DEM_EVENT_STATUS_FAILED; DEM_EVENT_STATUS_PASSED;
Lin_SendFrame	DEM_EVENT_STATUS_FAILED; DEM_EVENT_STATUS_PASSED;

Chapter 7 Function Calls

7.1 Function Calls during Startup

LIN shall be initialized during STARTUP phase of EcuM initialization. The API to be called for this is `Lin_Init()`. The MCU module should be initialized before the LIN is initialized. The LIN module shall be initialized by `Lin_Init(<&Lin_Configuration>)` service call during the start-up before the LIN peripherals are used. Please note that GPIO pins used for connection of LIN physical layer have to be properly assigned to desired FCUART module prior to the LIN initialization: so the MCU and PORT modules shall be initialized before LIN is initialized. After the LIN module is initialized, each LIN channel has to be initialized as well before using it. This is also done by the `Lin_Init(<&Lin_Configuration>)` service.

7.2 Function Calls during Shutdown

There is no shutdown specific procedure for Lin driver. LIN driver can go to sleep mode by using the following API services:

- `Lin_GoToSleepInternal(LIN_CHANNEL)`: which put the LIN driver into sleep mode without sending of Go-to-sleep command over the bus,
- `Lin_GoToSleep(LIN_CHANNEL)`: which put the LIN driver into sleep mode and send a Go-to-sleep command over the bus

7.3 Function Calls during Wake-up

The Lin driver supports external wake-up from the bus in 2 modes:

- If the channel is configured with "wake-up support", upon wakeup detection on Rx pin of the configured FCUART channel the ISR "`FCUARTx_RxTx_IRQHandler(LIN channel)`" will be executed (based on the LIN Channel configured) to wake-up Lin Driver.
- If the channel is not configured with "wake-up support", the Lin stack may call `Lin_CheckWakeup(LIN_CHANNEL)` service API in order to identify if a slave on the Lin bus issued a wake-up request. In case such a request is identified, then it is the Lin stack responsibility to wake up the channel and process the slave request.

Chapter 8 Other Requirements

8.1 Notification, Callback, Callout

- Notification

None

- Callback

There are no callback functions within the LIN driver.

- Callout

The LIN Driver uses 3 callouts functions which have to be provided by the respective module: `LinIf_WakeupConfirmation()`, `EcuM_SetWakeupEvent()` and `EcuM_CheckWakeup()` have to be provided by the EcuM module.

8.2 Macros

Please check various definitions available in Common module's include file `Mcal.h` for details.

- If OS is not used:

`AUTOSAR_OS_NOT_USED` needs to be defined.

In this case, If `USE_SW_VECTOR_MODE` is not defined, the user needs to map the interrupt vector table function to the ISR function, for example:

```
extern ISR(Lin_IsrTxRx_FCUART_0);

void FCUART0_RxTx_IRQHandler(void)

{

    Lin_IsrTxRx_FCUART_0 ();

}
```

- If OS is used:

Do not define `AUTOSAR_OS_NOT_USED`.

Chapter 9 Integration Steps

- 1) Configure the LIN module and generate configuration files (please refer to Building chapter for details).
- 2) Configure appropriate memory sections in linker file or other (please refer to Memory chapter for details).
- 3) Map interrupt notification to their vector locations (please refer to chapter ISR for details).
- 4) Build the LIN module with dependent modules.